

Automated Data Validation Pipeline

Engineering robust data quality gates and exception management frameworks in Python

Author / Engineer

R Pooja Sekhar

Repository Target

GitHub Portfolio Project (Python / Pandas)

1. Executive Summary & Objective

Data quality issues such as structural drift, missing values, duplicates, and statistical anomalies represent significant operational friction in modern data pipelines. Inaccurate data cascades into downstream analytics and machine learning training models, producing unreliable insights. This project builds a modular, object-oriented **Automated Data Validation Pipeline** using Python and Pandas to intercept defective records programmatically.

The primary objective is to enforce automated schema compliance, uniqueness checks, completeness bounds, and statistical outlier isolation before data reaches analytical stores or model training loops.

2. Technical Architecture & Modules

The validation pipeline decouples data quality checks into independent, pluggable functional modules:

Pipeline Module	Description & Operations
Module 1: Ingestion	Handles file parsing (CSV/Parquet) with secure exception handling for file corruption or I/O faults.
Module 2: Schema Validation	Verifies mandatory column presence and rigorous data type alignment against predefined expectations.
Module 3: Completeness	Scans designated critical attributes for null values and systemic data gaps.
Module 4: Uniqueness	Detects duplicate row entries and unique primary key identifier collisions.
Module 5: Outlier Detection	Applies Interquartile Range (IQR) mathematical bounds to isolate numerical anomalies.

3. Core Implementation Code

Below is the core production-grade class structure implemented in `validator.py` for the GitHub repository:

```
import logging import pandas as pd logging.basicConfig(level=logging.INFO, format="%(asctime)s
```

4. Repository Structure & Setup Guide

To publish and share this project effectively on GitHub, structure your repository directory cleanly:

- **`validator.py`**: The primary script containing the `DataValidator` class and execution workflow.
- **`requirements.txt`**: Lists exact dependencies (`pandas>=1.3.0`, `numpy>=1.20.0`).
- **`README.md`**: Comprehensive project documentation, installation steps, and usage examples.

5. Conclusion & Future Enhancements

This automated validation framework establishes a reliable first line of defense for data engineering workflows. Future iterations will incorporate distributed processing support via Apache Spark (PySpark) for multi-gigabyte datasets, configuration-driven YAML schema definitions, and automated webhook alerts for pipeline telemetry integration.